

Soluciones-junio-2018.pdf



Anónimo



Estructura de Computadores



2º Grado en Desarrollo de Videojuegos



Facultad de Informática
Universidad Complutense de Madrid



[Accede al documento original](#)

antes



**Descarga sin publi
con 1 coin**



Después

WUOLAH



Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

	ESTRUCTURA DE COMPUTADORES Examen Final - Junio de 2018
	Nombre _____ DNI _____ Apellidos _____ Grupo _____

1.- (4 puntos) Sea un procesador segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos. El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID para las instrucciones de salto, que se resuelven en dicha etapa.
- La detección de riesgos LDE y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera en la última etapa de cada unidad funcional.
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- No se implementan saltos retardados, sino que se busca siempre la siguiente instrucción al salto y se cancela su ejecución si el salto es tomado.
- Las unidades funcionales de las que dispone el procesador son las siguientes:

UF	Cantidad	Latencia	Segmentación
FP ADD	1	3	Sí
FP MUL	1	5	Sí
Int ALU	1	1	No

El siguiente código calcula los elementos del vector `double C[128]` a partir de los elementos de los vectores `double A[128]` y `double B[128]` (recuerda que un elemento en punto flotante de doble precisión ocupa 8 bytes). Los vectores se encuentran almacenados en memoria de forma consecutiva (A-B-C) y la dirección de comienzo del vector A es `0x80001000`.

LOOP:

```
LD    F2, 0(R6)
AND   R2, R4, R9
BEQ   R2, R0, ELSE
IF:
    ADDD F6, F2, F2
    MULD F6, F8, F6
    BEQ  R0, R0, END
ELSE:
    LD    F4, 1024(R6)
    MULD F6, F2, F4
    ADD   R10, R10, R2
    ADDD F10, F2, F4
END:
SD     F6, 2048(R6)
ADD    R6, R6, R3
ADD    R4, R4, R9
BNE    R4, R5, LOOP
SUB    R4, R4, R4
```

El contenido inicial de los registros es el siguiente:

R6=0x80001000
R3=8

WUOLAH

R5=128

R4=0

R9=1

R10=0

R7=-1

F8=5.5

El registro R0 siempre contiene un 0 en la arquitectura MIPS

- (2 puntos) Representa el diagrama instrucción-tiempo para la primera y segunda iteración e indica los cortocircuitos realizados. Indica claramente las paradas y sus causas.
- (1 punto) Calcula el CPI de la ejecución completa del código.
- (1 punto) Si el procesador utilizara saltos retardados, ¿qué instrucciones llevarías al *delay-slot* de cada uno de los saltos y qué efectos tendría?

Solución

a)

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41
LD	F	D	E	M	W																																				
AND		F	D	E	M	W																																			
BEQ			F	Dp	D	E	M	W																																	
ADDD				Fp	F	X	X	X	X																																
LD					F	D	E	M	W																																
MULD						F	Dp	D	T1	T2	T3	T4	T5	M	W																										
ADD						Fp	F	D	E	M	W																														
ADDD								F	D	A1	A2	A3p	A3	M	W																										
SD									F	Dp	Dp	D	Ep	E	M	W																									
ADD										Fp	Fp	F	Dp	D	E	M	W																								
ADD														Fp	F	D	E	M	W																						
BNE																F	Dp	D	E	M	W																				
SUB																	Fp	F	X	X	X	X																			
LD																		F	D	E	M	W																			
AND																			F	D	E	M	W																		
BEQ																				F	Dp	D	E	M	W																
ADDD																				Fp	F	D	E	M	W																
MULD																					F	Dp	Dp	D	T1	T2	T3	T4	T5	M	W										
BEQ																					Fp	Fp	F	D	E	M	W														
LD																						F	D	E	M	W															
SD																							F	Dp	Dp	D	E	M	W												
ADD																							Fp	Fp	F	D	E	M	W												
ADD																								F	D	E	M	W													
BNE																									F	Dp	D	E	M	W											
SUB																										Fp	F	X	X	X	X										

b)

1ª iteración completa: 1 al 23 = 23

3ª iteración: Igual que la primera – 4 = 19

5ª iteración: Igual que la 3ª

...

2ª iteración: 24 al 41 = 18

4ª iteración: Igual que la 2ª

...

Ciclos: $(23 + 19 \cdot 63) + 18 \cdot 64 = 2372$

Instrucciones: $11 \cdot 64 + 10 \cdot 64 = 1344$

$CPI = 2372 / 1344 = 1.76$

c)

El ADD de R4 al hueco del primer BEQ. Aprovechamos el hueco y además evitamos la parada del salto BNE al alejar la dependencia.

BEQ R2, R0, ELSE

ADD R4, R4, R9

El primer MULD al hueco del segundo BEQ. Aprovechamos el hueco. Se aleja el MULD del ADDD pero, en cambio, se acerca el MULD al SD.

```
BEQ    R0, R0, END
MULD   F6, F8, F6
```

El ADD de R6 al hueco del salto BNE. Aprovechamos el hueco. Se acerca el ADD al LD del comienzo del LOOP, pero la dependencia se resuelve por forwarding sin generar parada.

```
BNE    R4, R5, LOOP
ADD    R6, R6, R3
```

2. (3 puntos) En un sistema con procesador ARM y una memoria de 64KB se quiere analizar el comportamiento del siguiente fragmento de un programa.

```
.global start
.equ N, 12

.data
A: .word N valores separados por comas
B: .word N valores separados por comas

.bss
C: .space 4*N

.text
....
mov R3, #N
mov R4, #0
mov R5, #0
mov R2, #0
for: cmp R2, R3
     bge fin
     ldr R4, [R0, R2, LSL#2]
     ldr R5, [R1, R2, LSL#2]
     add R4, R4, R5
     str R4, [R7, R2, LSL#2]
     add R2, R2, #1
     b for
fin:
```

1. Asumiendo que el programa se enlaza ubicando la sección .data a partir de la **dirección 0x0090** y la sección. bss a continuación de la sección .data, y que el fragmento de la sección .text en la que se encuentra la primera instrucción (mov R3, #N) se ubica en la **dirección 0x0140**, y que R0, R1 y R7 contienen las direcciones de A, B y C respectivamente; responde de forma razonada a las siguientes preguntas:

- (0.5 puntos) Obtener el rango de direcciones que ocupa el fragmento de programa y cada uno de los arrays. ¿Cuántos bloques de memoria principal ocupan? ¿Qué bloques son?
- (1.5 puntos) El sistema dispone de caches separadas de datos (D\$) e instrucciones (I\$) de 32 bytes cada una, ambas con política de emplazamiento directo, y políticas de escritura directa (write through) con asignación en escritura (write allocate), y tamaño de bloque de 16 bytes. Obtener los fallos que se producirán en la cache de datos (D\$) y en la cache de instrucciones (I\$) al ejecutar el fragmento del programa. Indicar, además del número total de fallos, para qué bloques de memoria principal se producen los fallos.
- (1 punto) Ahora añadimos al sistema de memoria una cache de segundo nivel unificada para datos e instrucciones (L2\$) de 128 bytes, asociativa por conjuntos de 2 vías, con reemplazamiento LRU y políticas de escritura directa (write through) con asignación en escritura (write allocate). Indicar el número de fallos que se producen en L2 tras ejecutar el fragmento del programa. (El tamaño de bloque es también de 16 bytes).

Solución:

- (0.5 puntos) Obtener el rango de direcciones que ocupa el programa con la tabla de literales, y cada uno de los arrays ¿Cuántos bloques de memoria principal ocupan? ¿Qué bloques son?

Programa = 12 instrucciones: Ocupan tres Bloques de MP: $B_{MP} 20$, $B_{MP} 21$, $B_{MP} 22$,
Rango de direcciones: 0x0140 - 0x 017F

Array A: Ocupan dos Bloques de MP: $B_{MP} 9$, $B_{MP} 10$ y $B_{MP} 11$ Rango de direcciones: 0x0090 - 0x 00BF

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio



Necesito concentración

ali ali oohh esto con 1 coin me lo quito yo...

WUOLAH

Array B: Ocupan dos Bloques de MP: B_{MP} 12, B_{MP} 13 y B_{MP} 14, Rango de direcciones: 0x00C0 - 0x 00EF

Array C: Ocupan dos Bloques de MP: B_{MP} 15, B_{MP} 16 y B_{MP} 17, Rango de direcciones: 0x00F0 - 0x 011F

b) (1.5 puntos)

11 1 4

| Etiqueta | Bc | Posicion |

Si no tenemos en cuenta la segmentación

Ciclo	Cache	Bloque Mem	Bloque Cache
Fetch mov R3, #N	I\$	20	0
Fetch cmp R2, R3	I\$	21	1
Mem ldr R4, [R0, R2, LSL#2]	D\$	9	1
Mem ldr R5, [R1, R2, LSL#2]	D\$	12	0
Fetch add R4,R4,R5	I\$	22	0 (reemplazo B20)
Mem str R4, [R7, R2, LSL#2]	D\$	15	1 (reemplazo B9)
Mem ldr R4, [R0, R2, LSL#2]	D\$	9	1 (reemplazo B15)
Mem str R4, [R7, R2, LSL#2]	D\$	15	1 (reemplazo B9)
...			

Fallos en la I\$: 3

Fallos en la D\$: 2 por iteración (A y C) + 1 cada 4 iteraciones (B) = 24 + 3 = 27.

c) (1 puntos) Muestra la evolución del contenido de L2 (etiqueta y contenido de cada bloque) tras ejecutar el programa. ¿Cuántos fallos a L2 se producen tras ejecutar el programa completo?

12	4
B_{MP}	Byte del bloque

10	2	4
Etiqueta	Conj	Byte del bloque

11	1	4
Etiqueta	Bc	Byte del bloque

Ciclo	Bloque Mem	Conjunto Cache
Fetch mov R3, #N	20	0
Fetch cmp R2, R3	21	1

Mem ldr R4, [R0, R2, LSL#2]	9	1
Mem ldr R5, [R1, R2, LSL#2]	12	0
Fetch add R4,R4,R5	22	2
Mem str R4, [R7, R2, LSL#2]	15	3
Mem ldr R4, [R0, R2, LSL#2]	9	2
Mem str R4, [R7, R2, LSL#2]	15	3
...		

Como vemos, el bloque que contiene las instrucciones de la primera mitad del bucle va al mismo conjunto de cache que el bloque que contiene el primer bloque de A. Pero como hay 2 vías no hay reemplazo.

La correspondencia de bloques con conjunto de cache es la siguiente:

Bloque	Conjunto L2
9	1
10	2
11	3
12	0
13	1
14	2
15	3
20	0
21	1
22	2

Como vemos, hay algunos conflictos, pero nunca hay más de dos activos por conjunto. Por lo tanto, tendremos sólo los fallos de inicio en la cache (1 por bloque accedido): 12 fallos en total.

3.- En una empresa dedicada al análisis de datos tienen que utilizar un algoritmo que requiere sumar todos los elementos de cada columna de una matriz de 512x512 elementos. El código C de dicho algoritmo es el siguiente:

```
for (j = 0; j < 512; j++)
    for (i = 0; i < 512; i++)
        red[j] = red[j] + A[i][j];
```

Sabiendo que:

- En C los elementos de una matriz se emplazan en memoria consecutivos por filas
- El procesador es de 32 bits (direcciones virtuales de 32 bits).
- El array de enteros A[512][512] se ubica en el mapa virtual de memoria del proceso a partir de la dirección **0x00458000**, y el array de enteros red[512] se ubica justo a continuación de A.
- El sistema operativo (SO) maneja páginas de 4KB con reemplazo **LRU**.

se pide:

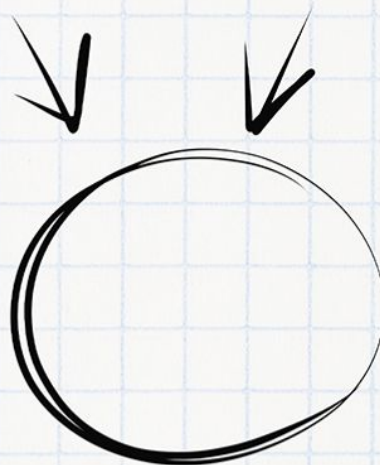
- (0.75 puntos)** Indicar qué páginas de memoria virtual ocupan los arrays A y red, indicando qué elementos de cada array están en cada página.
- (1.25 puntos)** El SO le asigna al proceso 8 marcos de página (lo que sería equivalente a decir que, desde el punto de vista de este proceso, la memoria física tiene 8 páginas). Sabiendo que 2 de esos 8 marcos están

Imagínate aprobando el examen

Necesitas tiempo y concentración

Planes	 PLAN TURBO	 PLAN PRO	 PLAN PRO+
 Descargas sin publi al mes	10 	40 	80 
 Elimina el video entre descargas			
 Descarga carpetas			
 Descarga archivos grandes			
 Visualiza apuntes online sin publi			
 Elimina toda la publi web			
 Precios Anual ☐	0,99 € / mes	3,99 € / mes	7,99 € / mes

Ahora que puedes conseguirlo,
¿Qué nota vas a sacar?



WUOLAH

ocupados por la página de código y la página de pila, accedidos constantemente y que por tanto nunca son elegidos para el reemplazo, y que los 6 restantes no contienen inicialmente ninguna página de A ni de red; indicar razonadamente cuántos fallos de página se producirán al ejecutar el código y el número de accesos a A y red que no suponen fallo de página.

- c) **(1 punto)** Razone si es posible o no reducir significativamente los fallos de página del proceso con alguna modificación en su código, indicando en su caso qué transformación se haría y cuantificando la mejora esperada.

Solución

a)

El array A está alineado con el tamaño de página.

El array red también, puesto que el tamaño de A es múltiplo del tamaño de página.

Cada fila de los arrays ocupa $512 \times 4B = 2KB$, media página, por tanto hay dos filas por página.

El array A ocupa por tanto 256 (0x100) páginas, de la 0x00458 a la 0x00557 ambas incluidas.

El array red ocupa la primera mitad de la página 0x00558.

En la página i [0-255] del array A (pág 0x00458 + i) están los elementos de las filas $2 \times i$ y $2 \times i + 1$.

En la única página de red están todos los elementos de red.

b)

Los 6 marcos van siendo ocupados por las páginas de datos. En la primera iteración, al referenciar $A[0][0]$ se produce un fallo de página y se traen las dos primeras filas de A. También se produce fallo al referenciar el elemento $red[0]$, y se trae la página que ocupa el array red. En la siguiente iteración no hay fallo ($A[1][0]$). En la tercera iteración se produce fallo al referenciar $A[2][0]$, pero no hay fallo al referenciar red.

Por tanto, al procesar la primera columna se producen $512/2 + 1$ fallos de página y 3×512 accesos a memoria. Se han traído 257 páginas, que han ido reemplazando a las últimas referenciadas usando los 6 marcos disponibles. Como la página usada para red se referencia en cada iteración, nunca es seleccionada para reemplazo.

Al procesar la siguiente columna, la página de red está cargada en un marco, pero las primeras páginas de A, ocupadas por las primeras filas, ya no están. Por tanto, se produce un fallo de página cada dos iteraciones. Esto se repite para cada columna.

Fallos de página: $512/2 \times 512 + 1 = 131073$.

Numero total de accesos a A y red: $512 \times 512 \times 3 = 786432$.

Número de accesos a A y red que no suponen fallo: $512 \times 512 \times 3 - (512/2 \times 512 + 1) = 655359$

c) El algoritmo permite un intercambio de bucles:

```
for (i = 0; i < 512; i++)  
    for (j = 0; j < 512; j++)  
        red[j] = red[j] + A[i][j];
```

En este caso, al comienzo de cada fila par se produce un fallo de página, se trae la página con las dos filas, se recorren todos sus elementos y ya no se vuelven a acceder. El número de fallos que se producirían sería de $512/2=256$, es decir, los fallos se reducen unas 512 veces.